Detecting SMS Spam Using Machine Learning and Deep Learning Approaches

MH4510 Group 7: Pattern Miners

Adele Teng, Clarrance Seah Jing Xiong, Wong Zi Xun, Kevin Lee Zhi Hong

22 November 2025

Abstract

This study implements machine learning and deep learning models for spam detection on a SMS Spam Collection Dataset. The dataset is a collection of SMS messages in English tagged with ham or spam labels. We applied the following models, Extreme Gradient Boosting (XGBoost) with BoW, LASSO with BoW and TF-IDF, LASSO with GloVe embeddings and Long Short-Term Memory (LSTM) Recurrent Neural Network with GloVE embeddings. The models' accuracy, precision, recall, F1-Score and AUC are compared. Results show that GloVE+LSTM performs the best. The findings provide a foundation for implementing effective feature engineering and diverse modeling techniques for high-performance spam detection systems.

Contents

1 Introduction **3**

1.1 Literature Review . . . 3

1.2 Research Objective . . . 3

2 Exploratory Data Analysis **3**

2.1 Dataset description . . . 3

2.2 Data cleaning and preprocessing . . . 3

2.3 Summary statistics and class imbalance . . . 4

2.4 Word frequency and word clouds . . . 4

3 Feature Engineering **4**

3.1 Bag-of-Words(BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) . . . 4

3.2 Global Vectors for Word Representation (GloVe) embeddings and sentence representations . . 5

4 Model Selection **6**

4.1 Extreme Gradient Boosting (XGBoost) . . . 6

4.2 LASSO logistic regression with BoW+TF-IDF . . . 6

4.3 LASSO with GloVe embeddings . . . 7

4.4 LSTM with GloVe embeddings . . . 7

5 Results and discussion **7**

5.1 Conclusion . . . 8

6 References **9**

7 Appendix **9**

1 Introduction

Spam detection for short message services (SMS) remains an important practical problem in natural language processing and cybersecurity. The objective of this project is to develop, compare, and critically evaluate several machine learning and deep learning approaches for classifying SMS messages as either ham (legitimate) or spam (unsolicited and often malicious). We use a popular SMS spam corpus and implement a complete pipeline that includes preprocessing, feature engineering, model training, and evaluation. The project aims to demonstrate differences in performance between classical models (e.g. XGBoost), sparse logistic methods (BoW+TF-IDF+LASSO), embedding-based linear models (GloVe + LASSO), and deep sequence models (GloVe + LSTM).

1.1 Literature Review

Automatic spam filtering is a long-studied problem in machine learning and information retrieval. Early systems relied heavily on Naïve Bayes classifiers and simple rule-based filters.(Androutsopoulos et. al, 2006) Later work introduced more sophisticated feature engineering (TF-IDF, n-grams) and ensemble methods such as gradient boosted trees (XGBoost).(Chen & Guestrin, 2016) Word embeddings such as word2vec and GloVe capture semantic relationships between words and have been used to generate dense sentence representations for downstream classification tasks.(Manning et. al, 2014) Recurrent neural networks, especially Long Short-Term Memory(LSTM) variants, are useful when sequential information (word order) is believed to be important. (Hochreiter & Schmidhuber, 1997)

Recent surveys emphasize that hybrid approaches which combine classical feature-based models and neural embeddings often yield robust performance on text classification benchmarks.(Wallacew & Zhang, 2018) Additionally, L1-regularized logistic regression (LASSO) is a popular and interpretable method to select a sparse set of predictive terms. (Tibshirani, 1996)

1.2 Research Objective

We aim to design and compare several models that can automatically classify SMS messages as ham (legitimate) or spam (malicious), using both classical machine learning and modern deep learning approaches. The models implemented in this study are Extreme Gradient Boosting (XGBoost) with BoW, LASSO with BoW and TF-IDF, LASSO with GloVe embeddings and Long Short-Term Memory (LSTM) Recurrent Neural Network with GloVE embeddings.

2 Exploratory Data Analysis

2.1 Dataset description

The dataset used in this study is a public SMS spam collection containing tagged SMS messages in English. Each row contains the message text and a label indicating ham or spam. After removing duplicates, the dataset consists of 5,169 messages.

2.2 Data cleaning and preprocessing

Preprocessing steps applied to the raw text include:

- Lowercasing text.
- Removing non-alphabetic characters (preserving "@" temporarily to mark emails).

- Replacing "@" with a token (e.g., twacc) to retain email occurrences.
- Lemmatization to improve NLP model performance.
- Tokenization for embedding and n-gram processing.

Lemmatization is a text-preprocessing technique that reduces words to their base form. For example, "walking" is reduced to "walk" and "better" to "good". Treating words with similar meanings as the same makes the text data more consistent, improving Natural Language Processing (NLP) Model performance. It also reduces data redundancy, leading to smaller datasets which are easier to handle for training machine learning models.

2.3 Summary statistics and class imbalance

Table 3.1 shows basic dataset statistics and class distribution.

Dataset summary (after cleaning)

Statistic	Value
Total messages	5169
Ham Proportion	87.4
Spam Proportion	12.6

The dataset highlights a significant class imbalance, with ham messages being the majority.

2.4 Word frequency and word clouds

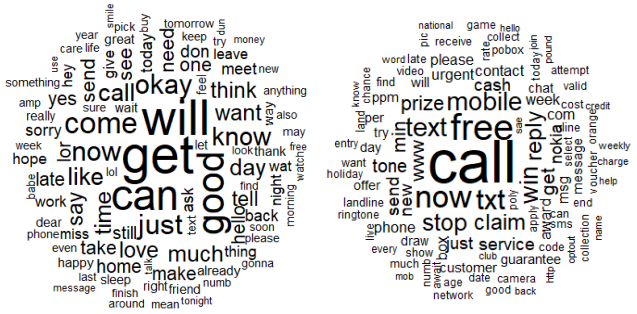

Figure 1: Ham Word Cloud (left), Spam Word Cloud (right)

3 Feature Engineering

The text data must be transformed into numerical vectors before implementing machine learning and deep learning models.

3.1 Bag-of-Words(BoW) and Term Frequency-Inverse Document Frequency (TF-IDF)

Bag-of-words model is a common method to transform NLP text data into numerical vectors. Words in the text data are considered as single tokens without considering the order of the words. A Document-Term-Matrix is created from the data, consisting of every word that appears in the data and their respective word occurrence.

As utilizing all unique vocabulary in the data to train the data is computationally expensive, words with occurrences of less than 10 are pruned from the data. This removes unsusual words that appear too rarely and hence will have little significance to the model, reducing the word count of the DTM from 5627 to 671.

R-code sample of the pruned vocabulary.

```
vocab
```

```
## Number of docs: 4136
## 0 stopwords:  ...
## ngram_min = 1; ngram_max = 1
## Vocabulary:
##                term term_count doc_count
##              <char>      <int>     <int>
##   1:         abiola         10        10
##   2:      camcorder         10        10
##   3:          clean         10         9
##   4:       complete         10        10
##   5: complimentary         10        10
##  ---
## 667:              s        456       406
## 668:           call        462       404
## 669:           will        475       427
## 670:            get        527       485
## 671:              u        895       634
```

A limitation of Bag-Of-Words is that it treats all words equally, hence words with high occurrences may dominate the document, and may not necessarily contribute meaningfully to the model. This can dilute the model's ability to focus on more meaningful terms.

TF-IDF mitigates this limitation by normalising the terms using the TD-IDF score, which is the product of Term Frequency(TF) and Inverse Document Frequency(IDF):

$$TF\text{-}IDF(t,d,D) = TF(t,d) \cdot IDF(t,D)$$

$$TF(t,d) = \log\left(\frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}\right), \ f_{t,d} = \text{raw count of term } t \text{ in document } d$$

$$IDF(t,D) = \log\left(\frac{N}{n_t}\right), \ D = \text{corpus/set of all documents},$$

$$N = \text{total number of documents in corpus}, \ n_t = \text{number of documents where term } t \text{ appears}$$

TD-IDF score balances both TF and IDF, highlighting terms that are both frequent within a specific document and distinctive across the corpus. Words that frequently appear in the document but are rare across the entire corpus are given more importance compared to in BoW. This adjusts word importance and helps to capture the significance of words in the context of the corpus.

3.2 Global Vectors for Word Representation (GloVe) embeddings and sentence representations

Another limitation of BoW is that it does not consider similarity between word vectors, such as between 'king' and 'queen', compared with 'keyboard'. GloVE, on the other hand, is an unsupervised learning algorithm that captures semantic relationships between words by analysing co-occurence patterns in the corpus.

A co-occurrence matrix is built to count the frequency of each word appearing near other words within a document. The algorithm minimises the difference between the dot product of word vectors and pointwise mutual information (PMI), which represents the expected co-occurrence probabilities. This adjusts the vectors such that the vectors will reflect the relationship between words based on the matrix.

Pointwise Mutual Information (PMI) is a measure of association between two events (in this case, words appearing in a text) based on joint probability of occurring, compared to their individual probabilities.

$$PMI(x, y) = \log\left(\frac{P(x, y)}{P(x) \cdot P(y)}\right),$$

$$P(x, y) = \text{joint probability of words x and y occuring together,}$$

$$P(x), P(y) = \text{individual probabilties of x and y respectively occuring independently}$$

High PMI value indicates strong association between the two words.

A pre-trained GloVe embedding is not used as the embeddings by Stanford were only trained on Wikipedia, Twitter, and Common Crawl, all of which fail to capture the extremely informal language that is often used in messages.

4 Model Selection

This section describes the models implemented, their motivations, and training details.

4.1 Extreme Gradient Boosting (XGBoost)

XGBoost is an ensemble learning method that implementsgradient boosted decision trees. Each successive tree sequentially fits the residuals of the previous trees, minimizing a loss function.

The XGBoost objective is logistic loss applied to a tree ensemble. If f_k denotes the k-th tree, predictions are:

$$\hat{y}_i = \sigma(\sum_{k=1}^{K} f_k(x_i)), \sigma(z) = \frac{1}{(1 + e^{-z})}$$

Key hyperparameters considered include max_depth (tree depth), eta(learning rate), subsample(fraction of training instances per tree) and colsample_bytree(fraction of data/features per tree). These are optimised via grid search combined with cross-validation, which ensures a balance between model performance and the risk of overfitting.

XGBoost is trained using a binary logistic objective and DTM features. Early stopping using a held-out validation set.

4.2 LASSO logistic regression with BoW+TF-IDF

LASSO (L1-penalized) logistic regression is applied to TF-IDF features to yield sparse predictive terms. L1 regularization enforces sparsity in model coefficients and highlights the most predictive terms. Cross-validation is used to select the regularization parameter λ. λ is tuned using cross-validation to optimize predictive performance while maintaining interpretability.

4.3 LASSO with GloVe embeddings

GloVe embeddings are fed into LASSO logistic regression as a low-dimensional representation of each message. This approach captures semantic similarity among words while maintaining interpretability through L1 regularization. The λ parameter is tuned via cross-validation similar to the TF-IDF model.

4.4 LSTM with GloVe embeddings

LSTM is a type of recurrent neural network designed to capture sequential dependencies in data. Each SMS message is represented as a sequence of word embeddings derived from GloVe. LSTM is designed to learn long-range patterns, with subsequent dense layers applied for classification. Key hyperparameters - such as the number of LSTM units, dropout rates, learning rate, and batch size - are chosen through a combination of standard heuristics to mitigate overfitting while maintaining predictive accuracy.

Document embeddings are structured such that each of the 100 GloVe-derived features is treated as a seperate timestep. THis sequence representation (21 timesteps $\times$ 100 feature per timestep) allows the LSTM to process feature dimensions sequentially. Dropout regularization and dense layers were incorporated to reduce overfitting risks.

The LSTM computes a state update and produces an output h_t. For binary classification the final output is passed through a sigmoid and trained with binary cross-entropy loss:

$$L = -\sum_{n=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

TensorFlow is an open-source library for numerical computation and machine learning, providing the backend for efficient tensor operations and automatic differentiation. It is used to provide robust support for building and implementing LSTM models.

Keras (and the updated keras3) is a high-level interface built on TensorFlow that simplifies the design, training, and evaluation of deep learning models, such as the LSTM used here. Keras allows rapid prototyping with layers, optimizers, and loss functions while delegating heavy computation to TensorFlow.

We acknowledge that hyperparameter tuning should be done when implementing LSTM, but as it is a very time-consuming process, we have chosen exclude hyperparameter tuning due to time constraints.

5 Results and discussion

Minimising false positives (misclassifying ham as spam) is critical to avoid classifying important emails as spam. However, because false negatives (misclassifying spam as ham) can also be harmful in light of security concerns and reducing scam victims, minimising false negatives is also integral to a model's performance in spam detection. Hence, our chosen evaluation metrics for the study - accuracy, precision, recall, F1-Score, specificity and AUC - consider how effective the models minimise both false postiive and false negatives.

Accuracy, precision and specificity are metrics that focus on the correctness of positive predictions, while recall is a metric that focus on how well the model captures all positive instances. F1-score balances precision and recall, while AUC is a performance metric that summarizes a model's ability to differentiate between them across all possible thresholds.

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**	**Specificity**	**AUC**
XGBoost	0.9699903	0.9541284	0.8	0.8702929	0.9944629	0.9762714
TF-IDF+LASSO	0.964182	0.9266055	0.7769231	0.8451883	0.9911406	0.9795979
GloVe+LASSO	0.9787028	0.9576271	0.8692308	0.9112903	0.9944629	0.9708749
GloVe+LSTM	0.9845111	0.9596774	0.9153846	0.9370079	0.9944629	0.9736903

While all models have a high accuracy and precision score, GloVE+LSTM has the highest with an accuracy of 0.984511 and precision of 0.9596774. GloVe+LSTM also has the highest recall, f1-Score and specificity (0.9370079, 0.9944629 and 0.9736903 respectively) amongst all the models.

The AUC of TF-IDF+LASSO is the highest amongst all the models, though it should be noted that the AUC of all 4 models are high and of similar value.

5.1 Conclusion

We conclude that GloVE+LSTM is the best model to use for spam detection due to having the highest overall performance across the evaluation metrics.

Some of the limitations of this study are:

- External validity. Model trained on one dataset might not generalize to other SMS datasets (different countries, languages, or spam campaigns).
- GloVe embedding was derived from the dataset instead of using a pre-trained GloVe embedding.

For future studies, training contextual embeddings like BERT, building an end-to-end sequence model with token indices and pretrained embeddings, and evaluating robustness across datasets and imbalanced learning strategies could improve models further for better performance.

6 References

- Androutsopoulos, I., Metsis, V., Paliouras, G. (2006). Spam filtering with naive Bayes – which naive Bayes? CEAS.
- Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
- Hochreiter, S.& Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.
- Manning, C. D., Pennington, J., Socher, R. (2014). GloVe: Global vectors for word representation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- SMS Spam Collection Dataset. (n.d.). [Dataset]. https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. Journal of the Royal Statistical Society: Series B (Methodological), 58(1), 267–288.
- Wallace, B. & Zhang, Y. (2018). A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification. Proceedings of the 8th International Joint Conference on Natural Language Processing (IJCNLP).

7 Appendix

Consolidated R Code for the report